

ECE-3103 INTEL 8086 MICROPROCESSOR

MD. EBTIDAUl KARIM

INTRODUCTION

- It is a 16 bit microprocessor.
- ALU works with 16 bit data.
- Data bus – 16 bit.
- Address bus – 20 bit. So it can address $2^{20} = 1048576$ memory locations. Each location of microprocessor is of 8 bit/ 1 byte. So 8086 microprocessor can address 1 MB
- Internal clock frequency of 8086 is 5 MHz, of 8086-1 is 10 MHz and of 8086-2 is 8 MHz.

INTRODUCTION

- ✓ It is a 40 pin dual-in-line chip.
- ✓ It uses a single +5V supply for its operations.

❑ Problems in 8085:

- In 8085 we can not perform various mathematical operation, such as: multiplication, division. It can perform addition, subtraction etc.
- 8085 is also unable to perform various trigonometric, exponential and complex functions.
- It can not also perform floating point operation.

INTRODUCTION

- Although 8086 is an improved version of 8085, it can not perform floating point operations, trigonometric, exponential and other complex functions.
- But 8086 can work with another processor, to perform above tasks. This co-processor is known as Math co-processor: Intel 8087.

INTERNAL STRUCTURE 8086

- 8086 cpu is divided into two independent functional parts.
 - a. Bus interface unit (BIU)
 - b. Execution unit (EU)
- BIU: The BIU handles all transactions of data and addresses on the buses for EU.
- ✓ The BIU performs all bus operations such as instruction **fetching**, reading and writing operands for memory and calculating the addresses of the memory operands.
- ✓ The instruction bytes are transferred to the instruction queue.

INTERNAL STRUCTURE 8086

❑ **EU:** EU executes instructions from the instruction system byte queue.

✓ Decodes instructions fetched by the BIU

✓ Generate control signals

✓ Executes instructions.

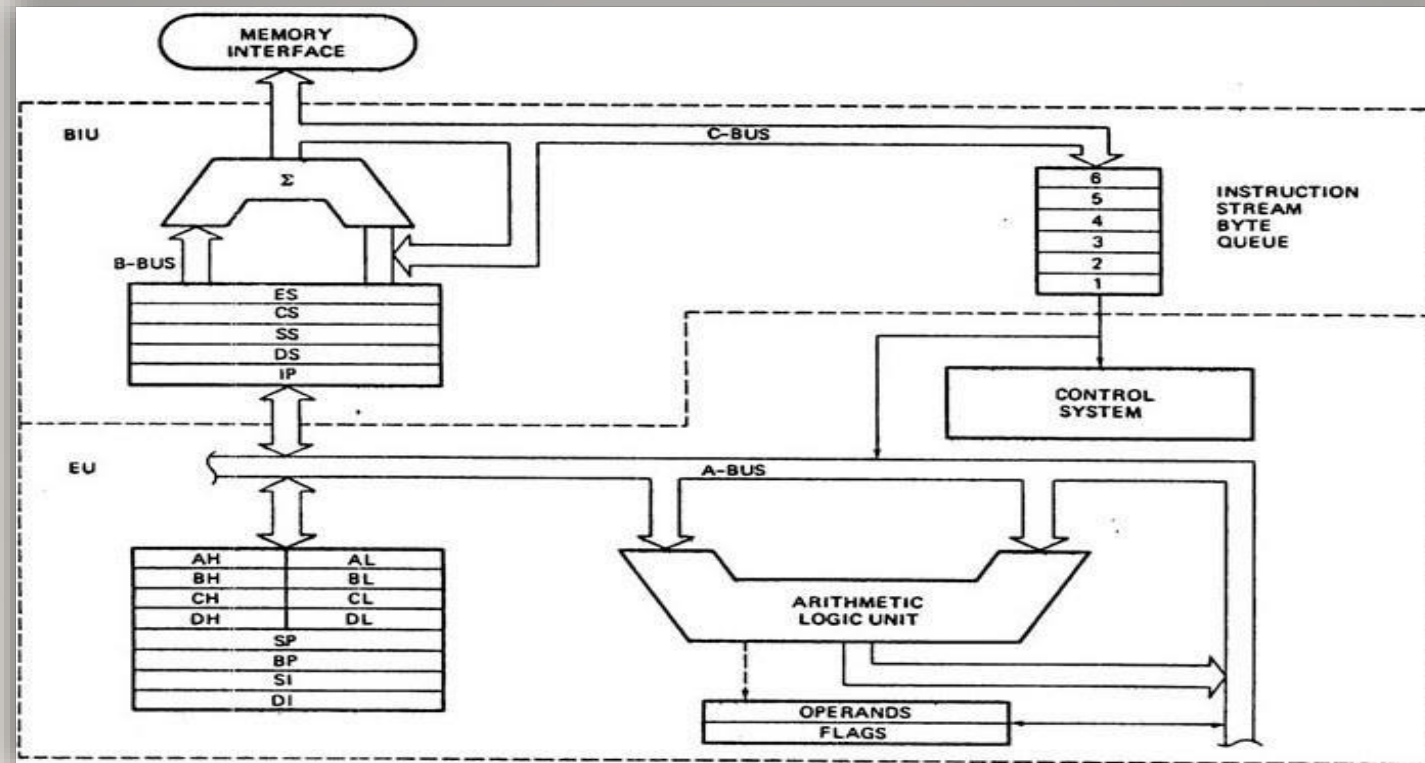
➤ BIU contains Instruction queue, Segment registers, Instruction pointer, Address adder.

➤ EU contains Control circuitry, Instruction decoder, ALU, Pointer and Index register, Flag register, General Purpose Registers.

INTERNAL STRUCTURE 8086

- So basic function of a microprocessor is to fetch, decode and execute an instruction.
- Fetch is done in BIU and decode and execution is done in EU.
- In EU, ALU performs AND, OR, XOR, NOT, INC, DEC, Shift etc. various operations.

INTERNAL STRUCTURE 8086



FLAG REGISTER

- The Flag register is a Special Purpose Register.
- A flag is a flip-flop that indicates some conditions produced by the execution of an instruction or controls certain operations of the EU.
- In 8086, flag register is of 16 bit. It contains nine active flags. Flag register is divided into two types: a) conditional flags(6) b) control flags(3)

FLAG REGISTER

- The 6 status flags are:
 - ✓ Sign Flag (S)
 - ✓ Zero Flag (Z)
 - ✓ Auxiliary Carry Flag (AC)
 - ✓ Parity Flag (P)
 - ✓ Carry Flag (CY)
 - ✓ Overflow Flag (O)

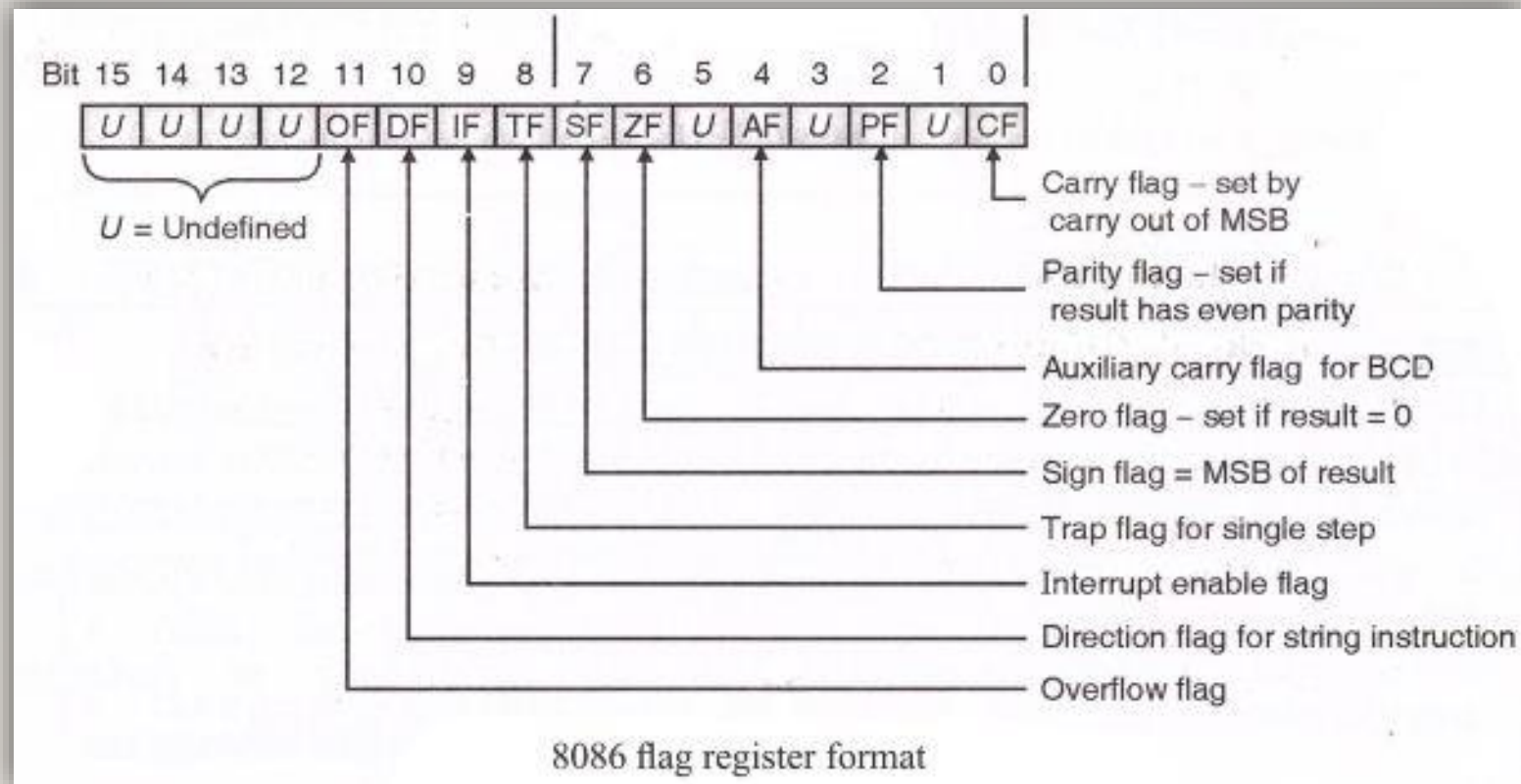
FLAG REGISTER

- **Control Flags** – The control flags enable or disable certain operations of the microprocessor.

There are 3 control flags in 8086 microprocessor and these are:

- ✓ Directional Flag (DF)
- ✓ Interrupt Flag (IF)
- ✓ Trap Flag (TF)

FLAG REGISTER



FLAG REGISTER

- **Overflow Flag (OF)** – This flag will be set (1) if the result of a signed operation is too large to fit in the number of bits available to represent it, otherwise reset (0).
- After any operation, if D[6] generates any carry and passes to D[7] OR if D[6] does not generate carry but D[7] generates, overflow flag becomes set, i.e., 1.
- If D[6] and D[7] both generate carry or both do not generate any carry, then overflow flag becomes reset, i.e., 0.

FLAG REGISTER

.

DEC

53

+91

144

hex

35

+5B

90

53=0 0 1 1 0 1 0 1

91=0 1 0 1 1 0 1 1

1 1 1 1 1 1 1 1

0 0 1 1 0 1 0 1

0 1 0 1 1 0 1 1

1 0 0 1 0 0 0 0 (144) [NEGATIVE]

CF=0 (no carry bit 6 to 7 and 7 to c) {carry
from bit 6 to 7 but no carry
from 7 to c }

OF=1 (we add two positive number but
result negative, so OF =1 because 144 is
outside of -128 to +127)



FLAG REGISTER

.		1 0 0 1 1 1 1 0
DEC	hex	<u>1 1 0 1 0 0 1 1</u>
-98	9E	1 0 1 1 1 0 0 0 1
-45	+D3 (ignore carry)	
<u>-143</u>	<u>171</u>	

CF=1 (no carry bit 6 to 7 but there is a carry from bit 7 to C)

OF=1 we add two negative number got positive !!! That must not be correct. so OF set to 1, because result 171 is outside the range 128 to +127) {no carry from bit 6 to 7 and carry from bit 7 to C OVERFLOW always 1 }

FLAG REGISTER

DEC	hex
53	35
-45	D3 (ignore carry)
<u>8</u>	<u>108</u>

53 = 0 0 1 1 0 1 0 1
- 45 = 1 1 0 1 0 0 1 1

0 0 1 1 0 1 0 1
1 1 0 1 0 0 1 1
1 0 0 0 0 1 0 0 0

CF = 1 (carry bit 6 to 7 and 7 to c)

OF = 0 ([redacted])

[redacted] OF = 0 because result 8 is between -128 to +127) {carry from bit 6 to 7 and carry from bit 7 to C OVERFLOW always 0 }

FLAG REGISTER

DEC

53

+25

78

hex

35

+19

4E

53=0 0 1 1 0 1 0 1

25=0 0 0 1 1 0 0 1

0 1 1 0 0 0 1

0 0 1 1 0 1 0 1

0 0 0 1 1 0 0 1

0 1 0 0 1 1 1 0 (78)

CF=0 (no carry bit 6 to 7 and 7 to c)

OF=0 (78 is between -128 to +127)

FLAG REGISTER

❑ **Directional Flag (D)** – This flag is specifically used in string instructions.

- ✓ If directional flag is set (1), then access the string data from higher memory location towards lower memory location.
- ✓ If directional flag is reset (0), then access the string data from lower memory location towards higher memory location.

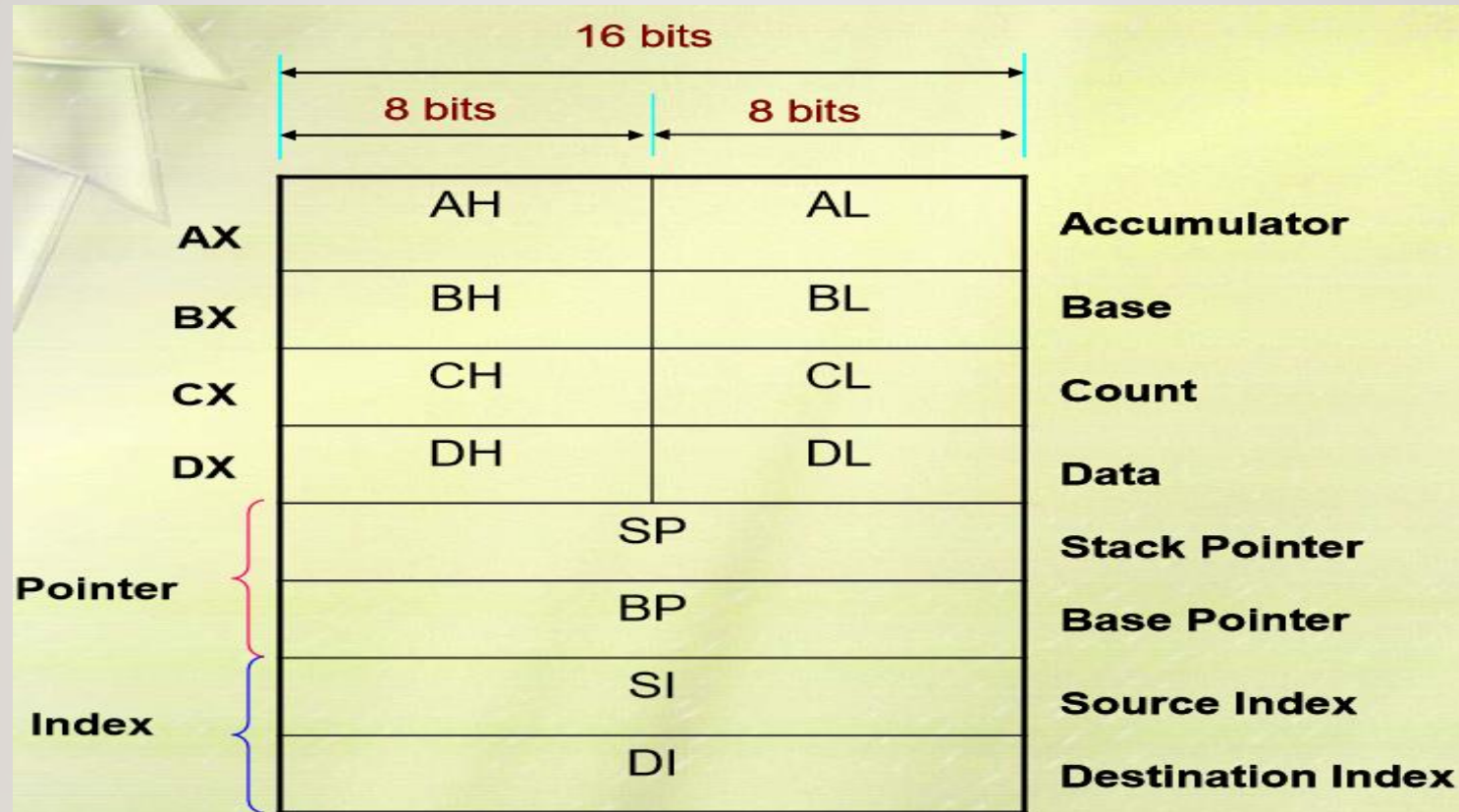
FLAG REGISTER

- . **Interrupt Flag (I)** – This flag is for interrupts.
- ✓ If interrupt flag is set (1), the microprocessor will recognize interrupt requests from the peripherals.
- ✓ If interrupt flag is reset (0), the microprocessor will not recognize any interrupt requests and will ignore them.
- The flag may be set or cleared using the CLI (**C**lear I**nterrupts**), STI (**S**et I**nterrupts**) and POPF (**P**op **F**lags) instructions.
- CLI clears IF (sets to 0), while STI sets IF to 1. POPF pops 16 bits off the stack into the FLAGS register, which means IF will be set or cleared based on the ninth bit on the top of the stack.

FLAG REGISTER

- **Trap Flag (T)** – This flag is used for on-chip debugging.
 - ✓ Setting trap flag puts the microprocessor into single step mode for debugging.
 - ✓ In single stepping, the microprocessor executes a instruction and enters into single step ISR.
 - ✓ If trap flag is set (1), the CPU automatically generates an internal interrupt after each instruction, allowing a program to be inspected as it executes instruction by instruction.
 - ✓ If trap flag is reset (0), no function is performed.

GENERAL PURPOSE RESISTORS, POINTERS AND INDEX



GENERAL PURPOSE RESISTORS

- 8086 has four 16 bit general purpose registers AX, BX, CX and DX. Store intermediate values during execution. Each of these have two 8 bit parts (higher and lower).
- **AX register:**
 - ✓ This is the accumulator. It is of 16 bits and is divided into two 8-bit registers AH and AL to also perform 8-bit instructions.
 - ✓ It is generally used for arithmetical and logical instructions but in 8086 microprocessor it is not mandatory to have accumulator as the destination operand.

GENERAL PURPOSE RESISTORS

- **BX register:**
 - ✓ This is the base register. It is of 16 bits and is divided into two 8-bit registers BH and BL to also perform 8-bit instructions.
 - ✓ It is used to store the value of the offset.
- **CX register:**
 - ✓ This is the counter register. It is of 16 bits and is divided into two 8-bit registers CH and CL to also perform 8-bit instructions.
 - ✓ It holds count for instructions like loop, rotate, shift and string operations.

GENERAL PURPOSE RESISTORS

- **DX register:**
 - ✓ This is the data register. It is of 16 bits and is divided into two 8-bit registers DH and DL to also perform 8-bit instructions.
 - ✓ It is used with AX to hold 32 bit values during multiplication and division.
 - ✓ If the result is more than 16-bits, the lower order 16-bits are stored in accumulator and higher order 16-bits are stored in DX register.

PIPELINING

- **Pipelining** is the process of accumulating and executing computer instructions and tasks from the processor via a logical pipeline. It allows storing, prioritizing, managing and executing tasks and instructions in an orderly process.
- The process of fetching the next instruction when the present instruction is being executed is called as pipelining.
- Pipelining has become possible due to the use of queue.
- BIU (Bus Interfacing Unit) fills in the queue until the entire queue is full. BIU restarts filling in the queue when at least two locations of queue are vacant.

PIPELINING

- The 8086 BIU will not initiate a fetch unless and until there are two empty bytes in its queue. 8086 BIU normally obtains two instruction bytes per fetch.
- When EU is busy in decoding and executing an instruction, the BIU fetches up to six instruction bytes for the next instructions.
- The size of any instruction of 8086 may vary from 1 to 6 bytes.
- These bytes are called as the pre-fetched bytes and they are stored in a first in first out (FIFO) register set, which is called as a queue.

ADVANTAGES OF PIPELINING

- The execution unit always reads the next instruction byte from the queue in BIU.
- This is faster than sending out an address to the memory and waiting for the next instruction byte to come.
- In short pipelining eliminates the waiting time of EU and speeds up the processing.

SEGMENT REGISTER

- 8086 BIU sends out 20-bit addresses, so it can addresses, any of 2^{20} or 1MB in memory.
- However at any given time 8086 works with only four 2^{16} or 64KB segments.
- 4 **segment registers** in the BIU are used to hold the upper 16 bits of the starting addresses of four memory segments that the 8086 is working with a particular time.
- The part of a segment starting address stored in a **segment register** is often called segment base.
- The BIU always inserts zeros for the lowest 4 bits(nibble) of the 20-bit starting address for a segment. For example if a code segment register contains 348AH, then code segment will start at address 348A0H.

SEGMENT REGISTER

❑ The four segment registers are:

- ✓ **Code segment(CS) register:** CS holds the base address for the Code Segment. All programs are stored in the Code Segment and accessed via the Instruction Pointer.
- ✓ **Stack Segment(SS) register:** A stack is a section of a memory set aside to store address and data while a subprogram executes. SS holds the base address for the Stack Segment.
- ✓ **Extra segment(ES) register:** ES holds the base address for the Extra Segment.
- ✓ **Data segment(DS) register:** DS holds the base address for the Data Segment.

SEGMENT REGISTERS

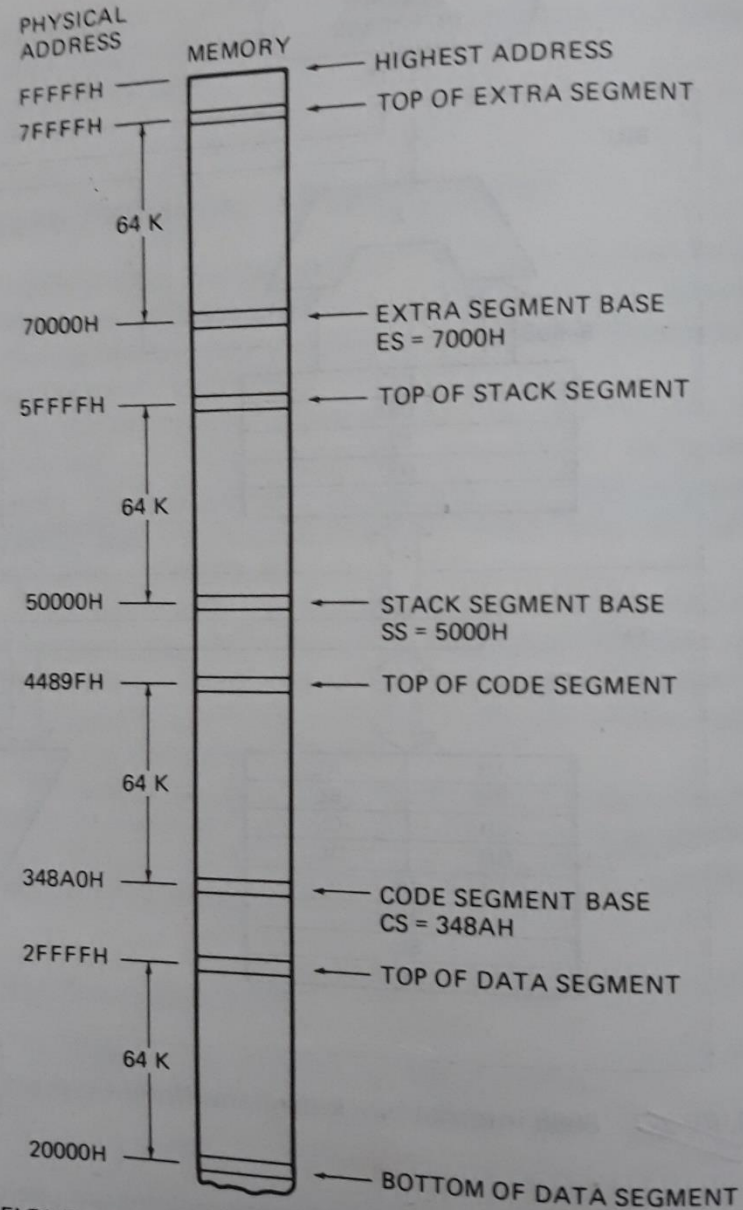


FIGURE 2-9 One way four 64-Kbyte segments might be positioned within the 1-Mbyte address space of an 8086.

Segment may overlap and start from any position

INSTRUCTION POINTER

- We know CS register holds the upper 16 bits of the starting address of the segment from which BIU is currently fetching instruction code bytes.
- Now instruction pointer(IP) is a 16 bit register. It holds 16-bit address or offset of the next instructions in the Code Segment.
- The value contained in IP is referred to as an offset because this value must be offset from (added to) the segment base address in CS to produce the required 20-bit physical address sent out by the BIU.
- CS is multiplied by 10H to give the 20 bit physical address of the Code Segment.

INSTRUCTION POINTER

- Address of the next instruction is calculated as $CS \times 10H + IP$.
- **Example:** $CS = 4321H$, $IP = 1000H$
- ✓ then $CS \times 10H = 43210H + \text{offset} = 44210H$
- Alternative way of representing 20-bit physical address is CS:IP or segment base: offset form
- IP is incremented after every instruction byte is fetched.
- IP gets a new value whenever a branch instruction occurs.

Instruction Pointer

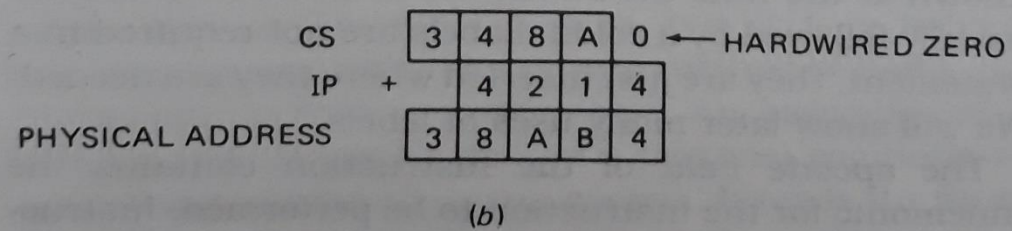
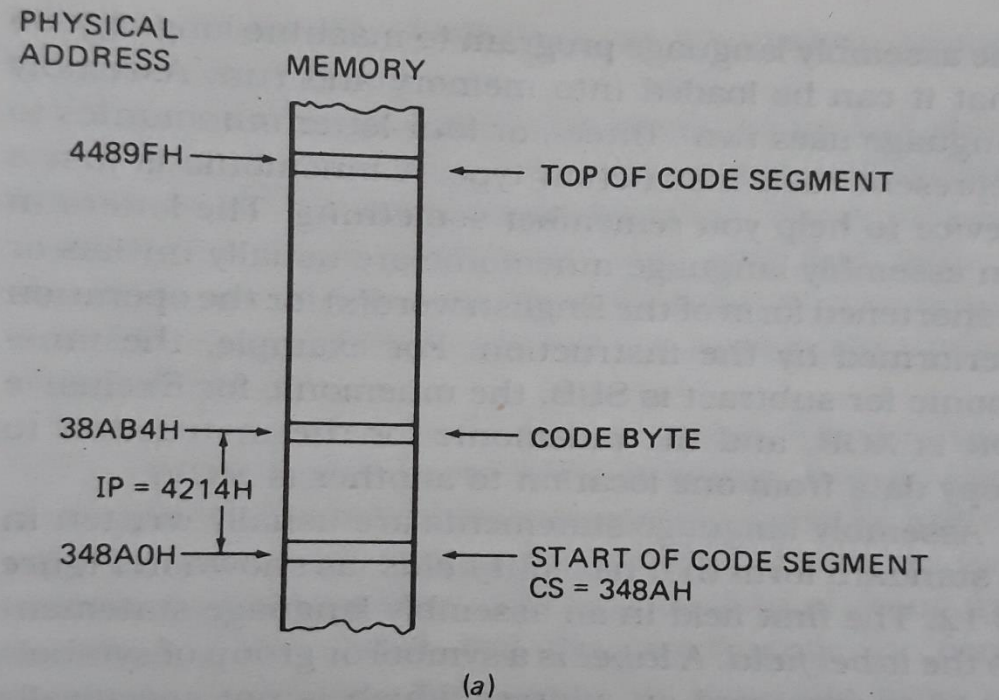


FIGURE 2-10 Addition of IP to CS to produce the physical address of the code byte. (a) Diagram. (b) Computation.

STACK POINTER

- A stack is a section of memory set aside to store addresses and data while a subprogram is executing. Upper 16-bits of the starting address for this segment are kept in the stack segment register.
- The Stack Pointer(SP) register in the execution unit holds the 16-bit offset from the start of the segment to the memory location where a word was most recently stored on the stack.
- The memory location where a word was most recently stored is called the **top of the stack**.
- Physical address for stack read or stack write is calculated as $SS \times 10H + SP$.
- EU contains the stack pointer.

STACK POINTER

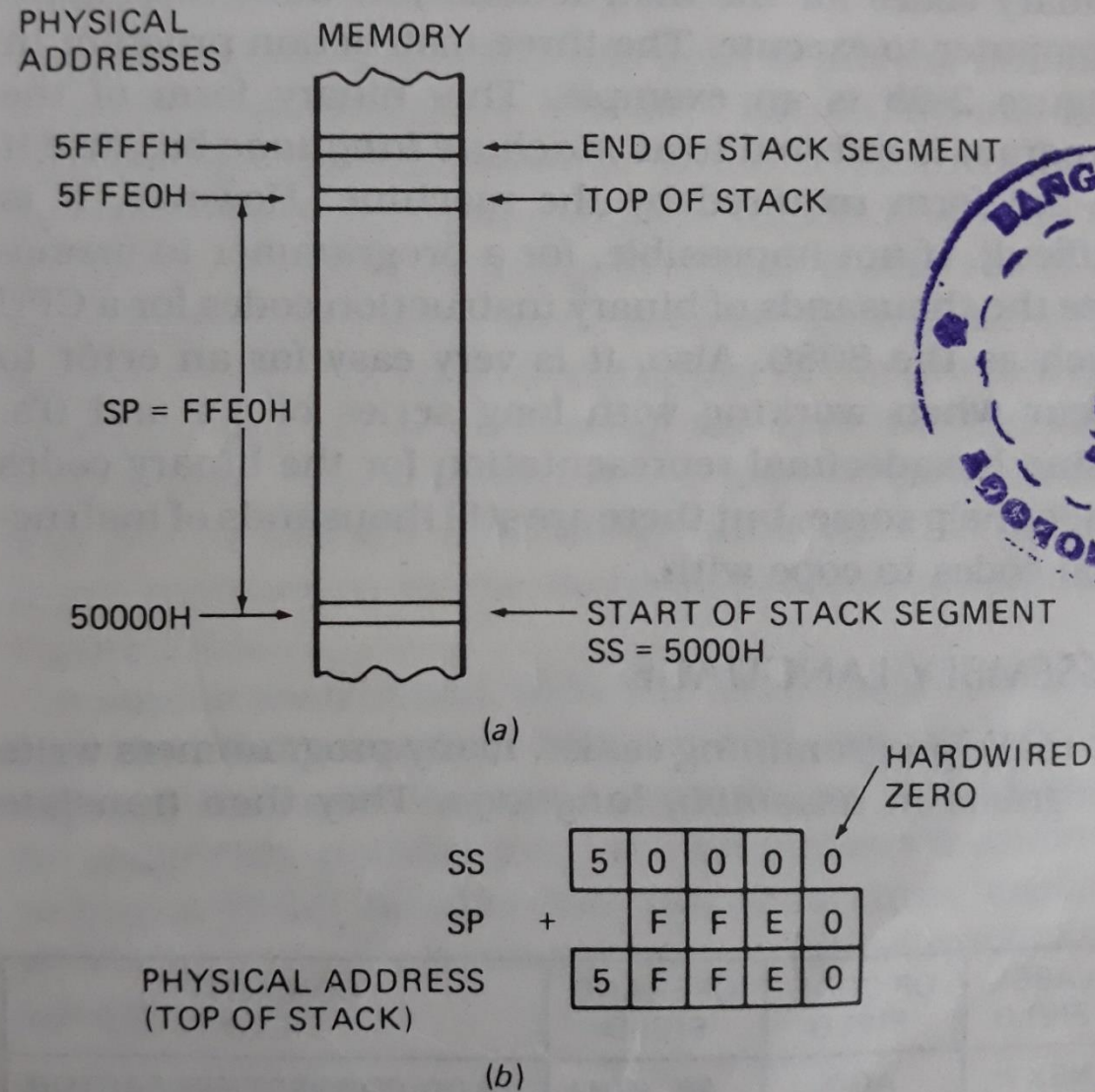


FIGURE 2-11 Addition of SS and SP to produce the physical address of the top of the stack. (a) Diagram. (b) Computation.

POINTER AND INDEX REGISTERS

- **BP** – This is the base pointer. It is of 16 bits.
- ✓ BP can hold offset address of any location in the stack segment. It is used to access random locations of the stack.
- **SI: Source Index register:** is required for some string operations
- ✓ When string operations are performed, the SI register points to memory locations in the data segment which is addressed by the DS register. Thus, SI is associated with the DS in string operations.

POINTER AND INDEX REGISTERS

- **DI: Destination Index register** – is also required for some string operations.
- ✓ When string operations are performed, the DI register points to memory locations in the extra segment which is addressed by the ES register. Thus, DI is associated with the ES in string operations.

CONTROL SECTION AND OPERAND

❑ Control Section:

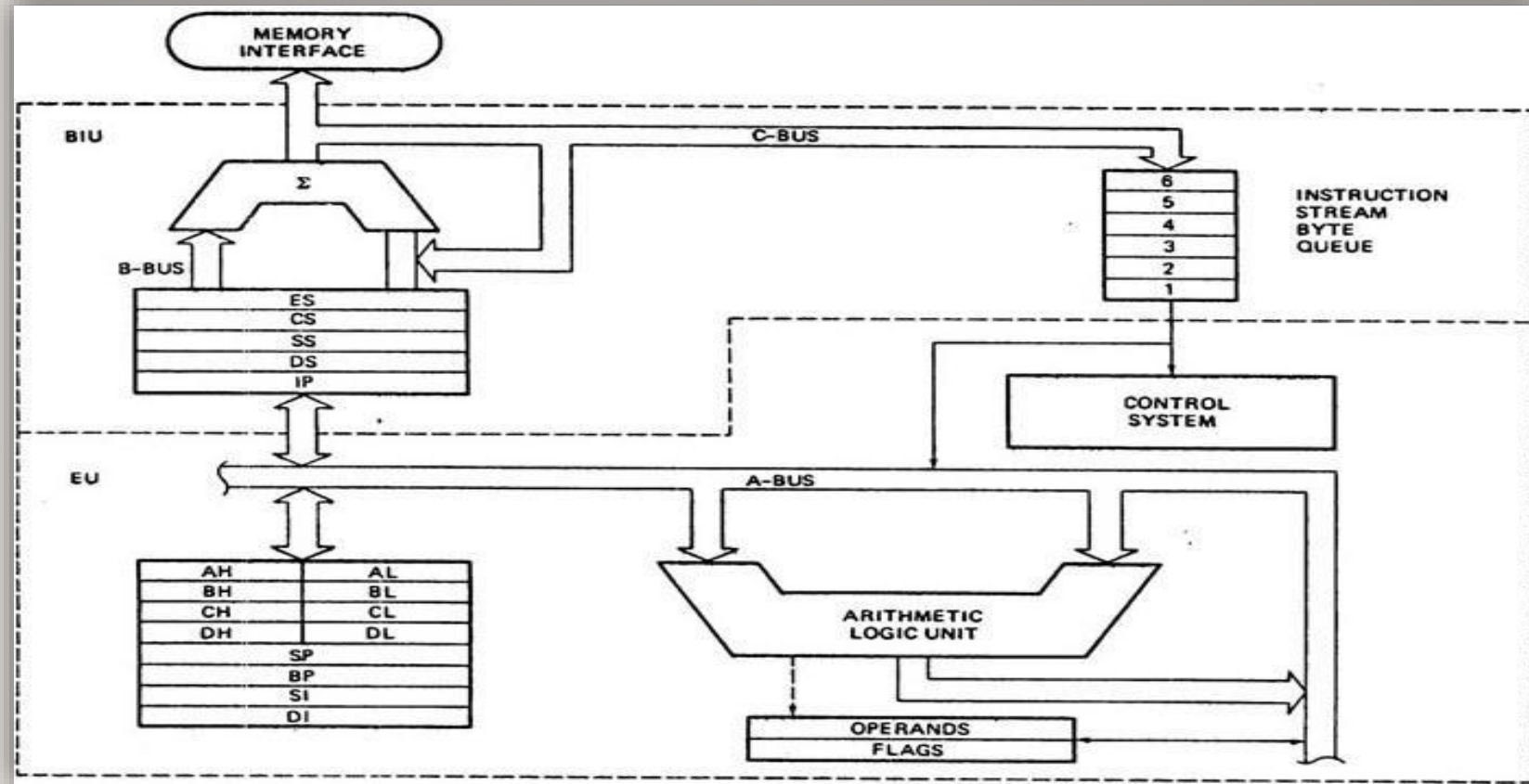
- The instructions that are fetched by BIU are decoded in control section and control section sends necessary control signals to various parts of EU for the execution of the program.
- It tells each register to perform various functions at particular time.

❑ Operands: It is used by 8086 as a temporary register. For example: XCHG BX,CX

- ✓ In this operation at first the value stored in CX is stored in operand. When we move value of BX to CX, then value at operand is stored to BX register.



INTERNAL STRUCTURE 8086-FINAL



Thank You